AI & Software Engineering Workshop

Week 1 — Day 1: Setup and First Message

Edik Simonian, Summer 2026

What you'll build this week

- A **Telegram bot** powered by a real LLM
- With its own **personality** — you design it
- Custom **slash commands** — you code them
- **Memory** that survives restarts
- **Live on the internet** by Friday — running even when your laptop is closed

By the end of today: the bot replies to you, from code on your machine.

What is a bot?

Two ways Telegram talks to your code:

- **Polling** — your code keeps asking Telegram *"any new messages?"*
  → what we use **today**, on your laptop
- **Webhook** — Telegram pushes each message to your server's URL
  → what we use **Friday**, in production

Same bot code either way — only the delivery changes.

The stack

Telegram → Flask (Python) → Cerebras → reply

- **Telegram Bot API** — the messaging interface
- **Flask** — receives the messages
- **Cerebras** — runs the LLM that writes the replies *(free tier)*
- **SQLite** — memory *(Day 4)*
- **PythonAnywhere** — hosting *(Day 5)*
- **GitHub** — your code, your tests, your deploys

Setup 1 of 3 — GitHub

1. Create a GitHub account *(if you don't have one)*
2. **Fork** the repo — your own copy, needed for auto-deploy on Day 5
3. Clone your fork and install:

```
git clone https://github.com/<your-username>/telegram-vercel-bot.git
cd telegram-vercel-bot
make install
```

Setup 2 of 3 — Telegram bot token

1. In Telegram, search for **@BotFather**
2. Send `/newbot`
3. Pick a name, then a username ending in `bot`
4. Copy the **token** — looks like `7123456789:AAF...`

Setup 3 of 3 — AI API key

1. Sign up at **cloud.cerebras.ai** *(free, no credit card)*
2. Profile icon → **API Keys** → **Create new API key**
3. Copy the key — looks like `csk-...`

Wire it up — `.env`

```
cp .env.example .env
```

Set two lines:

```
TELEGRAM_BOT_TOKEN=<your BotFather token>
AI_API_KEY=<your Cerebras key>
```

Leave the rest as-is.

> *Secrets live in `.env` — never in code, never in git.*

First contact

```
make run
```

```
Bot @your_bot_username starting in polling mode.
Send your bot a message on Telegram to try it out.
```

Message your bot. Watch the exchange appear in your terminal.

"Stateless mode" is expected — memory arrives on Day 4.

Prove it works — like an engineer

```
make test
```

- The whole suite runs **offline** — no API keys, no network
- The **same suite** runs in GitHub Actions on every push to your fork
- Green check on GitHub = your bot's logic still works

Your first code edit

`bot/handlers.py`:

```
@bot.message_handler(commands=["start"], func=is_allowed)
def cmd_start(message):
    bot.send_message(
        message.chat.id,
        "Hello! I'm your AI assistant...",
    )
```

Change the greeting → `Ctrl+C` → `make run` → send `/start`.

You just shipped your first change.

Today → Tomorrow

Today the bot works on your laptop and answers as a generic assistant.

Tomorrow: the most powerful single line in the project — the **system prompt**. Your bot gets a personality.